

LAB 7 : Graph Traversal Simulation & Coding

[CO3]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE ONE MENTIONED IN THE QUESTION.
- YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.

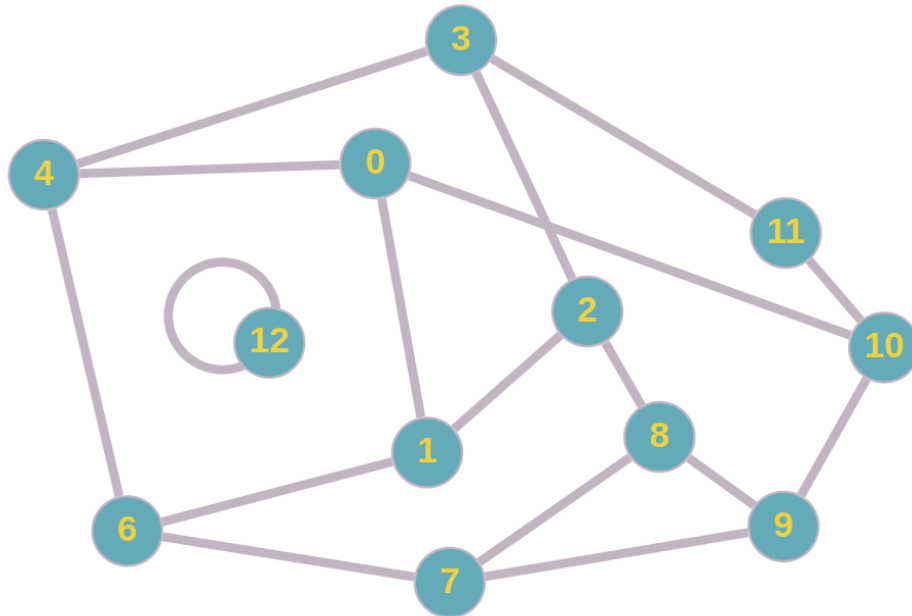
**Python List, Negative indexing and append() is
STRICTLY prohibited unless mentioned in the
question**

Lab Tasks: 4 tasks

SIMULATION TASKS:

Task 1: [Lab Task]

For the BFS Simulation refer to page 136 of [Chapter 7](#)



Simulate the BFS algorithm using a Queue choosing any specific vertex as a starting point.

Task 2: [Lab Task]

For the DFS Simulation refer to page 141 of [Chapter 7](#)

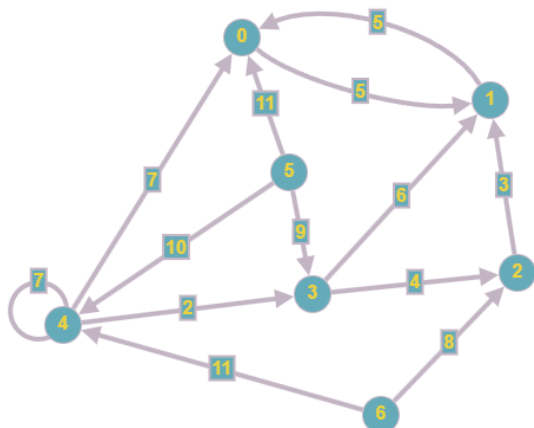
Simulate the DFS algorithm (including discovery time & finishing time) choosing any specific vertex as a starting point.

CODING TASKS:

[JAVA Template](#) & [Python Template](#)

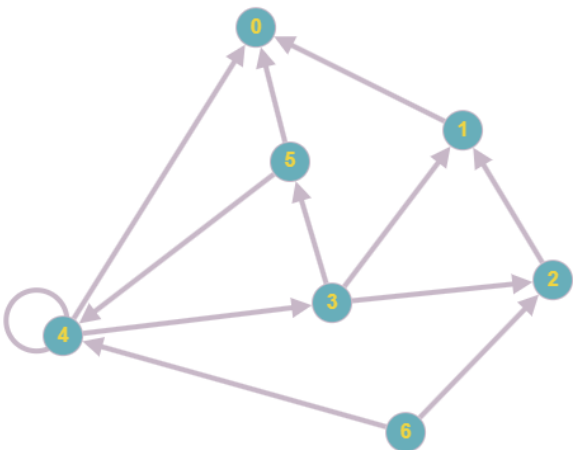
Task 1: [Lab Task]

Implement the BFS algorithm to traverse a Directed Weighted graph represented by Adjacency Matrix. Refer to **Page 134** of [Chapter 7](#).

SAMPLE GRAPH & Starting Vertex	SAMPLE OUTPUT																																																	
 <table><tr><td>0,</td><td>5,</td><td>0,</td><td>0,</td><td>0,</td><td>0,</td><td>0</td></tr><tr><td>5,</td><td>0,</td><td>0,</td><td>0,</td><td>0,</td><td>0,</td><td>0</td></tr><tr><td>0,</td><td>3,</td><td>0,</td><td>0,</td><td>0,</td><td>0,</td><td>0</td></tr><tr><td>0,</td><td>6,</td><td>4,</td><td>0,</td><td>0,</td><td>0,</td><td>0</td></tr><tr><td>7,</td><td>0,</td><td>0,</td><td>2,</td><td>7,</td><td>0,</td><td>0</td></tr><tr><td>11,</td><td>0,</td><td>0,</td><td>9,</td><td>10,</td><td>0,</td><td>0</td></tr><tr><td>0,</td><td>0,</td><td>8,</td><td>0,</td><td>11,</td><td>0,</td><td>0</td></tr></table> Starting Vertex: 6	0,	5,	0,	0,	0,	0,	0	5,	0,	0,	0,	0,	0,	0	0,	3,	0,	0,	0,	0,	0	0,	6,	4,	0,	0,	0,	0	7,	0,	0,	2,	7,	0,	0	11,	0,	0,	9,	10,	0,	0	0,	0,	8,	0,	11,	0,	0	BFS Traversal: At (6) Vertex -> -----> (2) is a new neighbor -----> (4) is a new neighbor At (2) Vertex -> -----> (1) is a new neighbor At (4) Vertex -> -----> (0) is a new neighbor -----> (3) is a new neighbor -----> (4) already known At (1) Vertex -> -----> (0) already known At (0) Vertex -> -----> (1) already known At (3) Vertex -> -----> (1) already known -----> (2) already known BFS Traversal Order: 6 2 4 1 0 3
0,	5,	0,	0,	0,	0,	0																																												
5,	0,	0,	0,	0,	0,	0																																												
0,	3,	0,	0,	0,	0,	0																																												
0,	6,	4,	0,	0,	0,	0																																												
7,	0,	0,	2,	7,	0,	0																																												
11,	0,	0,	9,	10,	0,	0																																												
0,	0,	8,	0,	11,	0,	0																																												
PseudoCode: visited = empty boolean array enqueue v //Here, v is the starting vertex mark v visited while queue is not empty: current = dequeue process(current) for each neighbor n of current: if n is not visited: enqueue n mark n visited																																																		
Note: The output will change depending on the graph & the starting point.																																																		

Task 2: [Lab Task]

Implement the DFS algorithm to traverse a Directed Unweighted graph represented by Adjacency Matrix. Refer to **Page 135** of [Chapter 7](#).

SAMPLE GRAPH & Starting Vertex	SAMPLE OUTPUT
 <pre> 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, </pre> Starting Vertex: 6	DFS Traversal: Visiting (6) Visiting (2) Visiting (1) Visiting (0) Visiting (4) Visiting (3) Visiting (5) DFS Traversal Order: 6 2 1 0 4 3 5
<u>PseudoCode:</u> <pre> //This function takes a starting point and only visits the reachable vertices DFS(Graph G, Vertex v, Visited) mark v as visited process(v) for each neighbor u of v in G if u is not visited DFS(G, u, Visited) </pre>	
<u>Note:</u> The output will change depending on the graph & the starting point.	
<u>Did you know?</u> DFS can also be implemented explicitly using a stack instead of recursion. Although both recursive DFS and stack-based DFS follow the same depth-first principle, the exact traversal order may differ slightly depending on how neighbors are inserted into and removed from the stack.	